

INTEGRATION GUIDE

# BAXS × T-0 Network

## 接口对接与 Sandbox

## 测试方案

基于 Connect RPC 的机构级清结算网络完整集成指南

发起方 | BAXS (baxs.ca)

对接平台 | T-0 Network (t-0.network)

文档版本 | v1.0

文档日期 | 2026年7月

保密级别 | 内部技术文档

# 目录

## 1. T-0 Network 平台概述

- 1.1 平台架构与角色定义
- 1.2 集成方式选择
- 1.3 环境信息

## 2. 认证机制

- 2.1 ECDSA 签名认证流程
- 2.2 HTTP 请求头规范
- 2.3 签名实现代码示例
- 2.4 公钥注册与管理

## 3. NetworkService API (BAXS 作为 OFI 调用)

- 3.1 GetQuote - 获取报价
- 3.2 CreatePayment - 创建支付
- 3.3 FinalizePayout - 确认付款结果
- 3.4 CompleteManualAmlCheck - AML 检查结果

## 4. ProviderService API (BAXS 作为 Provider 实现)

- 4.1 PayOut - 执行付款
- 4.2 UpdatePayment - 更新支付状态
- 4.3 UpdateLimit - 信用额度更新
- 4.4 AppendLedgerEntries - 账本条目
- 4.5 ApprovePaymentQuotes - 报价审批

## 5. 核心数据类型定义

- 5.1 金额与报价类型
- 5.2 区块链与稳定币枚举
- 5.3 Travel Rule (IVMS101) 数据

## 6. 幂等性实现规范

## 7. Sandbox 测试方案

7.1 测试环境配置

7.2 测试用例设计

7.3 端到端测试流程

8. 错误处理与监控

9. 实施路线图

10. 附录

# 1. T-0 Network 平台概述

## 1.1 平台架构与角色定义

T-0 Network 是由 Tether 战略支持的机构级稳定币清结算网络，基于 USDT 实现跨境支付的 near-instant 结算。网络采用 P2P 架构，连接全球持牌金融机构。

### 核心角色

表1-1 T-0 Network 角色定义

| 角色         | 英文                                         | 职责                         | BAXS 定位                      |
|------------|--------------------------------------------|----------------------------|------------------------------|
| 始发金融<br>机构 | Originating Financial<br>Institution (OFI) | 发起支付/换币请求，持有<br>USDT 并进行结算 | BAXS 作为 OFI 发起换币请<br>求       |
| 付款提<br>供商  | Payout Provider                            | 在目标国家执行本地法币付<br>款          | BAXS 同时作为 Provider<br>接收付款指令 |
| 收款提<br>供商  | Pay-In Provider                            | 接收 USDT 结算资金               | T-0 Network 上的对手方机<br>构      |
| 网络编<br>排层  | Network Orchestration                      | 报价匹配、支付路由、清结<br>算协调        | T-0 Network 核心服务             |

### BAXS 双角色定位

#### BAXS 在 T-0 Network 中的双重角色

**作为 OFI（始发方）：** BAXS 代表机构客户发起 USDT→法币的换币请求，调用 NetworkService API（GetQuote、CreatePayment、FinalizePayout）。

**作为 Provider（付款方）：** BAXS 接收来自 T-0 Network 的付款指令（PayOut），通过加拿大/美国/香港/新加坡的本地银行系统执行 CAD/USD/HKD/SGD 的法币付款，并通过 ProviderService 回调报告付款状态。

## 1.2 集成方式选择

T-0 Network 支持两种集成方式，均基于 Connect RPC 框架（同时支持 gRPC 和 REST/JSON 编码）：

表1-2 集成方式对比

| 维度    | SDK 方式（推荐）                       | 协议直连方式             |
|-------|----------------------------------|--------------------|
| 开发复杂度 | 低 - SDK 处理签名、验签、密钥管理             | 高 - 需自行实现所有加密操作    |
| 灵活性   | 中 - 受 SDK 更新节奏影响                 | 高 - 完全控制所有细节       |
| 维护成本  | 低 - SDK 自动更新协议变更                 | 高 - 需跟踪协议变更并手动更新   |
| 适用场景  | 快速上线、标准集成需求                      | 特殊需求、已有 gRPC 基础设施  |
| 支持语言  | Go, TypeScript, Python, Java, C# | 任何支持 gRPC/REST 的语言 |

**BAXS 推荐方案：**采用 SDK 方式进行初步集成，在独立 Adapter Service 中运行 SDK，与现有系统解耦。对于高性能场景（高并发报价流），可逐步迁移至协议直连方式。

## 1.3 环境信息

表1-3 T-0 Network 环境配置

| 环境         | API 端点                                                                        | 用途        | 区块链网络                       |
|------------|-------------------------------------------------------------------------------|-----------|-----------------------------|
| Production | <a href="https://api.t-0.network">https://api.t-0.network</a>                 | 生产环境真实交易  | Ethereum / Tron / BSC 主网    |
| Sandbox    | <a href="https://api-sandbox.t-0.network">https://api-sandbox.t-0.network</a> | 开发测试、集成验证 | 测试网（Goerli / Nile / Chapel） |

## 2. 认证机制

T-0 Network 所有通信均需使用 **ECDSA 签名 + Keccak-256 哈希** 进行认证，确保请求的身份验证和消息完整性。此方案与以太坊加密标准兼容。

### 2.1 ECDSA 签名认证流程

#### 签名过程

1. **获取时间戳**：获取当前 Unix 时间戳（毫秒），作为 64 位无符号整数，使用 little-endian 编码
2. **拼接数据**：将时间戳追加到请求体末尾
3. **哈希计算**：使用 Keccak-256 对拼接后的数据进行哈希
4. **签名生成**：使用 Provider 的 ECDSA 私钥对哈希值进行签名
5. **编码传输**：将签名以 hex 编码放入 HTTP Header

#### 验证过程（反向操作）

1. 从 x-Signature-Timestamp 提取时间戳
2. 将时间戳追加到请求体
3. 使用 Keccak-256 哈希拼接数据
4. 使用已知公钥验证签名

### 2.2 HTTP 请求头规范

每个请求必须包含以下三个认证 Header：

表2-1 认证 HTTP Header 规范

| Header       | 格式                   | 说明                                |
|--------------|----------------------|-----------------------------------|
| x-Signature  | Hex-encoded ECDSA 签名 | 请求体 + 时间戳的 Keccak-256 哈希的签名       |
| x-Public-Key | Hex-encoded 公钥       | 压缩格式(33 bytes)优先，也支持非压缩(65 bytes) |

## 2. 认证机制

X-Signature-Timestamp

Unix 时间戳(毫秒)

用于防重放攻击，网络验证窗口为  $\pm 1$  分钟

---

**安全要求：**时间戳必须在当前时间的  $\pm 1$  分钟窗口内，否则请求被拒绝。这有效防止重放攻击。Provider 必须拒绝任何签名验证失败的请求。

### 2.3 签名实现代码示例

## TypeScript / Node.js 实现

```

import { createHash, randomBytes } from 'crypto';
import { ecsign, pubToAddress, toCompactSig } from 'ethereumjs-util';
import keccak256 from 'keccak256';

class T0Authenticator {
  private privateKey: Buffer;
  private publicKey: Buffer;

  constructor(privateKeyHex: string) {
    this.privateKey = Buffer.from(privateKeyHex.replace('0x', ''), 'hex');
    // 从私钥派生公钥 (使用 secp256k1)
    this.publicKey = /* 派生逻辑 */;
  }

  /**
   * 对请求进行签名
   */
  signRequest(body: string | Buffer): {
    signature: string;
    publicKey: string;
    timestamp: number;
  } {
    // 1. 获取当前时间戳 (毫秒)
    const timestamp = Date.now();

    // 2. 时间戳编码为 64 位无符号整数 little-endian
    const timestampBuffer = Buffer.allocUnsafe(8);
    timestampBuffer.writeBigUInt64LE(BigInt(timestamp));

    // 3. 拼接请求体和时间戳
    const bodyBuffer = Buffer.isBuffer(body) ? body : Buffer.from(body);
    const combined = Buffer.concat([bodyBuffer, timestampBuffer]);

    // 4. Keccak-256 哈希
    const hash = keccak256(combined);

    // 5. ECDSA 签名
    const sig = ecsign(hash, this.privateKey);
    const signature = toCompactSig(sig.v, sig.r, sig.s).toString('hex');
  }
}

```



```

import { createHash, randomBytes } from 'crypto';
import { ecdsign, pubToAddress, toCompactSig } from 'ethereumjs-util';
import keccak256 from 'keccak256';

class T0Authenticator {
  private privateKey: Buffer;
  private publicKey: Buffer;

  constructor(privateKeyHex: string) {
    this.privateKey = Buffer.from(privateKeyHex.replace('0x', ''), 'hex');
    // 从私钥派生公钥 (使用 secp256k1)
    this.publicKey = /* 派生逻辑 */;
  }

  /**
   * 对请求进行签名
   */
  signRequest(body: string | Buffer): {
    signature: string;
    publicKey: string;
    timestamp: number;
  } {
    // 1. 获取当前时间戳 (毫秒)
    const timestamp = Date.now();

    // 2. 时间戳编码为 64 位无符号整数 little-endian
    const timestampBuffer = Buffer.allocUnsafe(8);
    timestampBuffer.writeBigUInt64LE(BigInt(timestamp));

    // 3. 拼接请求体和时间戳
    const bodyBuffer = Buffer.isBuffer(body) ? body : Buffer.from(body);
    const combined = Buffer.concat([bodyBuffer, timestampBuffer]);

    // 4. Keccak-256 哈希
    const hash = keccak256(combined);

    // 5. ECDSA 签名
    const sig = ecdsign(hash, this.privateKey);
    const signature = toCompactSig(sig.v, sig.r, sig.s).toString('hex');

    return {
      signature: `0x${signature}`,
    };
  }
}

```

## Go 实现

```

package auth

import (
    "crypto/ecdsa"
    "encoding/binary"
    "encoding/hex"
    "time"

    "github.com/ethereum/go-ethereum/crypto"
)

type Authenticator struct {
    privateKey *ecdsa.PrivateKey
    publicKey  []byte
}

func NewAuthenticator(privateKeyHex string) (*Authenticator, error) {
    pk, err := crypto.HexToECDSA(privateKeyHex)
    if err != nil {
        return nil, err
    }
    return &Authenticator{
        privateKey: pk,
        publicKey:  crypto.CompressPubkey(&pk.PublicKey),
    }, nil
}

func (a *Authenticator) SignRequest(body []byte) (signature, publicKey string,
timestamp int64, err error) {
    timestamp = time.Now().UnixMilli()

    // 时间戳编码为 little-endian
    tsBuf := make([]byte, 8)
    binary.LittleEndian.PutUint64(tsBuf, uint64(timestamp))

    // 拼接并哈希
    combined := append(body, tsBuf...)
    hash := crypto.Keccak256(combined)

    // ECDSA 签名

```

```

package auth

import (
    "crypto/ecdsa"
    "encoding/binary"
    "encoding/hex"
    "time"

    "github.com/ethereum/go-ethereum/crypto"
)

type Authenticator struct {
    privateKey *ecdsa.PrivateKey
    publicKey  []byte
}

func NewAuthenticator(privateKeyHex string) (*Authenticator, error) {
    pk, err := crypto.HexToECDSA(privateKeyHex)
    if err != nil {
        return nil, err
    }
    return &Authenticator{
        privateKey: pk,
        publicKey:  crypto.CompressPubkey(&pk.PublicKey),
    }, nil
}

func (a *Authenticator) SignRequest(body []byte) (signature, publicKey string,
timestamp int64, err error) {
    timestamp = time.Now().UnixMilli()

    // 时间戳编码为 little-endian
    tsBuf := make([]byte, 8)
    binary.LittleEndian.PutUint64(tsBuf, uint64(timestamp))

    // 拼接并哈希
    combined := append(body, tsBuf...)
    hash := crypto.Keccak256(combined)

    // ECDSA 签名
    sig, err := crypto.Sign(hash, a.privateKey)
    if err != nil {

```

## 2.4 公钥注册与管理

在接入 T-0 Network 之前，BAXS 需要生成 ECDSA 密钥对并将公钥注册到网络。每个 Provider 可注册多个公钥以支持密钥轮换。

密钥管理策略：

1. 生成：使用 HSM 或 AWS KMS 生成 secp256k1 密钥对
2. 注册：通过 T-0 Network 管理后台注册公钥
3. 轮换：定期生成新密钥对，逐步替换旧密钥
4. 撤销：密钥泄露时立即在 T-0 Network 撤销公钥
5. 备份：使用 Shamir Secret Sharing (3-of-5) 分片备份私钥

推荐工具：

- AWS KMS ( asymmetric ECC\_SECG\_P256K1 )
- HashiCorp Vault ( Transit secrets engine )
- Thales Luna HSM

### 3. NetworkService API（BAXS 作为 OFI 调用）

NetworkService 是 T-0 Network 对外暴露的核心服务接口，BAXS 作为 OFI 通过调用这些 API 来发起换币交易。所有方法均为幂等方法，可安全重试。

表3-1 NetworkService 方法概览

| 方法                     | 请求类型                          | 响应类型                           | 模式 |
|------------------------|-------------------------------|--------------------------------|----|
| UpdateQuote            | UpdateQuoteRequest            | UpdateQuoteResponse            | ST |
| GetQuote               | GetQuoteRequest               | GetQuoteResponse               | U  |
| CreatePayment          | CreatePaymentRequest          | CreatePaymentResponse          | U  |
| FinalizePayout         | FinalizePayoutRequest         | FinalizePayoutResponse         | U  |
| CompleteManualAmlCheck | CompleteManualAmlCheckRequest | CompleteManualAmlCheckResponse | U  |

#### 3.1 GetQuote - 获取报价

请求指定币种和金额的最优报价。系统会在所有可用的 Provider 报价中选择最优方案，同时考虑汇率竞争力和信用额度。

## 请求: GetQuoteRequest

表3-2 GetQuoteRequest 字段定义

| 字段       | 类型            | 必填 | 说明                              |
|----------|---------------|----|---------------------------------|
| amount   | PaymentAmount | 是  | 支付金额 (payout 或 settlement 金额)   |
| currency | string        | 是  | 目标 payout 货币代码 (如 "CAD", "HKD") |

## 响应: GetQuoteResponse

响应为 oneof 类型, 可能返回成功报价或失败原因:

```
// 成功响应
GetQuoteResponse.Success {
    quote_id: QuoteId          // 报价唯一标识
    payout_amount: Decimal      // 目标法币金额
    settlement_amount: Decimal  // USDT 结算金额
    provider_id: uint32         // 选中的 Provider ID
    expiration: Timestamp       // 报价过期时间
    quote_type: QuoteType       // 报价类型
}

// 失败响应
GetQuoteResponse.Failure {
    reason: enum {
        REASON_UNSPECIFIED = 0
        REASON_NO_QUOTE_AVAILABLE = 1    // 无可报价
        REASON_LIMIT_EXCEEDED = 2        // 信用额度不足
        REASON_CURRENCY_NOT_SUPPORTED = 3 // 不支持的货币
        REASON_INVALID_AMOUNT = 4        // 无效金额
    }
}
```

## REST/JSON 调用示例

```
// Request
POST https://api-sandbox.t-0.network/tzero.v1.payment.NetworkService/GetQuote
Content-Type: application/json
X-Signature: 0x...
X-Public-Key: 0x...
X-Signature-Timestamp: 17200000000000

{
  "amount": {
    "value": {
      "unscaled": "50000",
      "exponent": 0
    }
  },
  "currency": "HKD"
}

// Response - Success
{
  "success": {
    "quoteId": { "value": "quote_abc123" },
    "payoutAmount": { "unscaled": "390000", "exponent": 0 },
    "settlementAmount": { "unscaled": "50000", "exponent": 0 },
    "providerId": 42,
    "expiration": "2026-07-01T12:05:00Z"
  }
}
```

## 3.2 CreatePayment - 创建支付

提交换币付款请求。可指定 quote\_id 锁定特定报价，或不指定由系统自动选择最优报价。

请求: CreatePaymentRequest

表3-3 CreatePaymentRequest 字段定义

| 字段                | 类型     | 必填 | 说明                    |
|-------------------|--------|----|-----------------------|
| payment_client_id | string | 是  | BAXS 生成的唯一付款标识（用于幂等性） |

|                  |                |   |                       |
|------------------|----------------|---|-----------------------|
| amount           | PaymentAmount  | 是 | 支付金额                  |
| currency         | string         | 是 | 目标 payout 货币          |
| payment_details  | PaymentDetails | 是 | 收款方银行账户信息             |
| quote_id         | QuoteId        | 否 | 指定报价 ID (不指定则自动选择)    |
| travel_rule_data | TravelRuleData | 是 | FATF Travel Rule 合规数据 |

## TravelRuleData 结构

```
CreatePaymentRequest.TravelRuleData {
    originator: repeated ivms101.Person          // 发起方信息 (自然人或法人)
    beneficiary: repeated ivms101.Person          // 受益方信息
    originator_provider_legal_entity_id: uint32 (optional) // 发起 Provider 法人实体 ID
}
```

## 响应: CreatePaymentResponse

oneof 响应类型:

```
// 接受
CreatePaymentResponse.Accepted {
    payment_id: uint64          // 网络分配的付款 ID
    settlement_amount: Decimal  // 结算金额
    payout_amount: Decimal      // 付款金额
    payout_provider_id: uint32   // 选中的 Provider ID
}

// 失败
CreatePaymentResponse.Failure {
    reason: enum {
        REASON_UNSPECIFIED = 0
        REASON_NO_QUOTE_AVAILABLE = 1
        REASON_LIMIT_EXCEEDED = 2
        REASON_INVALID_QUOTE_ID = 3          // 无效的 quote_id
        REASON_QUOTE_EXPIRED = 4             // 报价已过期
        REASON_INVALID_PAYMENT_DETAILS = 5 // 付款详情无效
    }
}
```



## REST/JSON 调用示例

POST https://api-sandbox.t-0.network/tzero.v1.payment.NetworkService/CreatePayment

```
{
  "paymentClientId": "baxs_pay_20260701_001",
  "amount": {
    "value": { "unscaled": "50000", "exponent": 0 }
  },
  "currency": "HKD",
  "paymentDetails": {
    "beneficiary": {
      "name": { "name": "ABC Trading Corp" },
      "account": { "accountNumber": "1234567890" },
      "accountType": "ACCOUNT_TYPE_BUSINESS"
    },
    "paymentMethod": { "type": "PAYMENT_METHOD_BANK_TRANSFER" }
  },
  "quoteId": { "value": "quote_abc123" },
  "travelRuleData": {
    "originator": [{
      "naturalPerson": {
        "name": {
          "nameIdentifiers": [{
            "legalPersonName": "ABC Trading Corp",
            "legalPersonNameIdentifierType": "LEGL"
          }]
        },
        "geographicAddress": [{
          "addressType": "GEOG",
          "townName": "Vancouver",
          "country": "CA"
        }]
      }
    }],
    "beneficiary": [{
      "legalPerson": {
        "name": {
          "nameIdentifiers": [{
            "legalPersonName": "ABC Trading Corp",
            "legalPersonNameIdentifierType": "LEGL"
          }]
        }
      }
    ]
  }
}
```

POST https://api-sandbox.t-0.network/tzero.v1.payment.NetworkService/CreatePayment

```
{
  "paymentClientId": "baxs_pay_20260701_001",
  "amount": {
    "value": { "unscaled": "50000", "exponent": 0 }
  },
  "currency": "HKD",
  "paymentDetails": {
    "beneficiary": {
      "name": { "name": "ABC Trading Corp" },
      "account": { "accountNumber": "1234567890" },
      "accountType": "ACCOUNT_TYPE_BUSINESS"
    },
    "paymentMethod": { "type": "PAYMENT_METHOD_BANK_TRANSFER" }
  },
  "quoteId": { "value": "quote_abc123" },
  "travelRuleData": {
    "originator": [{
      "naturalPerson": {
        "name": {
          "nameIdentifiers": [{
            "legalPersonName": "ABC Trading Corp",
            "legalPersonNameIdentifierType": "LEGL"
          }]
        },
        "geographicAddress": [{
          "addressType": "GEOG",
          "townName": "Vancouver",
          "country": "CA"
        }]
      }
    ]},
    "beneficiary": [{
      "legalPerson": {
        "name": {
          "nameIdentifiers": [{
            "legalPersonName": "ABC Trading Corp",
            "legalPersonNameIdentifierType": "LEGL"
          }]
        }
      }
    ]}
  ]
}
```

### 3.3 FinalizePayout - 确认付款结果

报告付款的最终执行结果（成功或失败），替代已废弃的 ConfirmPayout 方法。

请求: FinalizePayoutRequest

表3-4 FinalizePayoutRequest 字段定义

| 字段         | 类型                            | 说明                  |
|------------|-------------------------------|---------------------|
| payment_id | uint64                        | 网络分配的 payment_id    |
| success    | FinalizePayoutRequest.Success | 付款成功（与 failure 二选一） |
| failure    | FinalizePayoutRequest.Failure | 付款失败（与 success 二选一） |

```
FinalizePayoutRequest.Success {
    receipt: PaymentReceipt (optional) // 付款凭证/收据
}

FinalizePayoutRequest.Failure {
    reason: enum {
        REASON_UNSPECIFIED = 0
        REASON_INSUFFICIENT_FUNDS = 1
        REASON_ACCOUNT_NOT_FOUND = 2
        REASON_ACCOUNT_FROZEN = 3
        REASON_INVALID_ACCOUNT_DETAILS = 4
        REASON_AML_REJECTED = 5
        REASON_PROVIDER_ERROR = 6
        REASON_NETWORK_ERROR = 7
        REASON_MANUAL_AML_CHECK_REQUIRED = 8
    }
}
```

### 3.4 CompleteManualAmlCheck - AML 检查结果

当 Provider 返回 ManualAmlCheck 响应时，BAXS 完成人工 AML 审核后通过此 API 报告结果。

```
CompleteManualAmlCheckRequest {
    payment_id: uint64
    approved: CompleteManualAmlCheckRequest.Approved    // 批准 (空消息)
    rejected: CompleteManualAmlCheckRequest.Rejected     // 拒绝
}

CompleteManualAmlCheckRequest.Rejected {
    reason: string    // 拒绝原因
}

CompleteManualAmlCheckResponse {
    approved: CompleteManualAmlCheckResponse.Approved    // 含更新后的金额和报价
    rejected: CompleteManualAmlCheckResponse.Rejected    // 拒绝确认
}
```

# 4. ProviderService API (BAXS 作为 Provider 实现)

ProviderService 是 BAXS 需要实现的服务端接口，T-0 Network 会主动调用这些接口向 BAXS 发送付款指令和状态更新。所有方法必须实现为幂等方法。

表4-1 ProviderService 方法概览

| 方法                   | 请求类型                       | 响应类型                        | 模式    |
|----------------------|----------------------------|-----------------------------|-------|
| PayOut               | PayoutRequest              | PayoutResponse              | UNARY |
| UpdatePayment        | UpdatePaymentRequest       | UpdatePaymentResponse       | UNARY |
| UpdateLimit          | UpdateLimitRequest         | UpdateLimitResponse         | UNARY |
| AppendLedgerEntries  | AppendLedgerEntriesRequest | AppendLedgerEntriesResponse | UNARY |
| ApprovePaymentQuotes | ApprovePaymentQuoteRequest | ApprovePaymentQuoteResponse | UNARY |

### 4.1 PayOut - 执行付款

T-0 Network 调用此方法请求 BAXS 执行法币付款。BAXS 收到后通过本地银行系统（加拿大 Interac/EFT、美国 Wire/ACH、香港 RTGS/CHATS、新加坡 FAST/MEPS）向收款方付款。

请求: PayoutRequest

表4-2 PayoutRequest 字段定义

| 字段               | 类型                           | 说明               |
|------------------|------------------------------|------------------|
| payment_id       | uint64                       | 网络分配的付款 ID       |
| payout_id        | uint64                       | 本次 payout 的唯一 ID |
| amount           | Decimal                      | 付款金额             |
| currency         | string                       | 付款货币             |
| payment_details  | PaymentDetails               | 收款方信息            |
| travel_rule_data | PayoutRequest.TravelRuleData | Travel Rule 数据   |

## 响应: PayoutResponse

```
// 接受 - BAXS 确认执行付款
PayoutResponse.Accepted { }

// 失败 - 无法执行付款
PayoutResponse.Failed {
  reason: enum {
    REASON_UNSPECIFIED = 0
    REASON_INSUFFICIENT_FUNDS = 1
    REASON_ACCOUNT_NOT_FOUND = 2
    REASON_ACCOUNT_FROZEN = 3
    REASON_INVALID_ACCOUNT_DETAILS = 4
    REASON_LIMIT_EXCEEDED = 5
    REASON_CURRENCY_NOT_SUPPORTED = 6
    REASON_PROVIDER_ERROR = 7
  }
}

// 需要人工 AML 审核
PayoutResponse.ManualAmlCheck { }
```

## BAXS 服务端实现示例 (TypeScript/Node.js)

```
import { ConnectRouter } from '@connectrpc/connect';
import { ProviderService } from '../gen/provider_pb';

const payoutStore = new Map<string, PayoutResponse>();

export default (router: ConnectRouter) =>
  router.service(ProviderService, {
    async payOut(request) {
      const key = request.paymentId.toString();

      // 幂等性检查: 已处理则返回原结果
      if (payoutStore.has(key)) {
        return payoutStore.get(key)!;
      }

      // 1. 验证收款账户信息
      const validation = await validatePaymentDetails(
        request.paymentDetails
      );
      if (!validation.valid) {
        const resp = { failed: { reason: validation.reason } };
        payoutStore.set(key, resp);
        return resp;
      }

      // 2. AML 自动检查
      const amlResult = await autoAmlCheck(request);
      if (amlResult.requiresManualReview) {
        const resp = { manualAmlCheck: {} };
        payoutStore.set(key, resp);
        return resp;
      }

      // 3. 提交本地银行付款
      const bankResult = await submitLocalBankTransfer({
        amount: request.amount,
        currency: request.currency,
        beneficiary: request.paymentDetails.beneficiary,
        region: determineRegion(request.currency),
      });
```



```
import { ConnectRouter } from '@connectrpc/connect';
import { ProviderService } from '../gen/provider_pb';

const payoutStore = new Map<string, PayoutResponse>();

export default (router: ConnectRouter) =>
  router.service(ProviderService, {
    async payOut(request) {
      const key = request.paymentId.toString();

      // 幂等性检查: 已处理则返回原结果
      if (payoutStore.has(key)) {
        return payoutStore.get(key)!;
      }

      // 1. 验证收款账户信息
      const validation = await validatePaymentDetails(
        request.paymentDetails
      );
      if (!validation.valid) {
        const resp = { failed: { reason: validation.reason } };
        payoutStore.set(key, resp);
        return resp;
      }

      // 2. AML 自动检查
      const amlResult = await autoAmlCheck(request);
      if (amlResult.requiresManualReview) {
        const resp = { manualAmlCheck: {} };
        payoutStore.set(key, resp);
        return resp;
      }

      // 3. 提交本地银行付款
      const bankResult = await submitLocalBankTransfer({
        amount: request.amount,
        currency: request.currency,
        beneficiary: request.paymentDetails.beneficiary,
        region: determineRegion(request.currency),
      });

      if (bankResult.success) {
```

## 4.2 UpdatePayment - 更新支付状态

T-0 Network 调用此方法通知 BAXS 支付状态变更 (成功或失败)。

```
UpdatePaymentRequest {
    payment_id: uint64
    accepted: UpdatePaymentRequest.Accepted      // 支付被接受
    confirmed: UpdatePaymentRequest.Confirmed    // 支付已确认完成
    failed: UpdatePaymentRequest.Failed          // 支付失败
    manualAmlCheck: UpdatePaymentRequest.ManualAmlCheck // 需人工 AML 审核
}

UpdatePaymentRequest.Accepted { }
UpdatePaymentRequest.Confirmed { }
UpdatePaymentRequest.ManualAmlCheck { }

UpdatePaymentRequest.Failed {
    reason: enum {
        REASON_UNSPECIFIED = 0
        REASON_NO_QUOTE_AVAILABLE = 1
        REASON_LIMIT_EXCEEDED = 2
        REASON_INVALID_QUOTE_ID = 3
        REASON_QUOTE_EXPIRED = 4
        REASON_INVALID_PAYMENT_DETAILS = 5
    }
}
```

## 4.3 UpdateLimit - 信用额度更新

T-0 Network 推送信用额度和已使用额度信息, 帮助 BAXS 实时了解可用额度。

```
UpdateLimitRequest {  
  limits: repeated UpdateLimitRequest.Limit  
}  
  
UpdateLimitRequest.Limit {  
  correspondent_id: uint32      // 对手方 ID  
  currency: string             // 货币代码  
  limit: Decimal               // 总信用额度  
  usage: Decimal               // 已使用额度  
}  
  
UpdateLimitResponse { }
```

## 4.4 AppendLedgerEntries - 账本条目

T-0 Network 推送账本更新，用于追踪 BAXS 在网络的财务敞口和交易流水。

```
AppendLedgerEntriesRequest {
  transactions: repeated Transaction
}

AppendLedgerEntriesRequest.Transaction {
  transaction_id: uint64      // 递增交易 ID (可能有间隔)
  entries: repeated LedgerEntry
}

AppendLedgerEntriesRequest.LedgerEntry {
  account_owner_id: uint32    // 1=网络账户, 其他=参与者 ID
  account_type: AccountType   // 账户类型
  debit: Decimal              // 借方金额 (贷方为 0)
  credit: Decimal             // 贷方金额 (借方为 0)
}

// 交易类型枚举
AppendLedgerEntriesRequest.AccountType {
  ACCOUNT_TYPE_UNSPECIFIED = 0
  ACCOUNT_TYPE_PAYOUT = 1      // 付款
  ACCOUNT_TYPE_SETTLEMENT = 2  // 结算
  ACCOUNT_TYPE_FEE = 3         // 费用
  ACCOUNT_TYPE_NETWORK_FEE = 4 // 网络费
  ACCOUNT_TYPE_RESERVE = 5     // 预留
}

AppendLedgerEntriesResponse { }
```

## 4.5 ApprovePaymentQuotes - 报价审批 (Last Look)

在人工 AML 审核完成后, T-0 Network 调用此方法让 BAXS 审批最终的付款报价。

#### 4. ProviderService API (BAXS 作为 Provider 实现)

```
ApprovePaymentQuoteRequest {
    payment_id: uint64
    quote_id: QuoteId
    settlement_amount: Decimal
    payout_amount: Decimal
}

ApprovePaymentQuoteResponse {
    accepted: ApprovePaymentQuoteResponse.Accepted
    rejected: ApprovePaymentQuoteResponse.Rejected
}

ApprovePaymentQuoteResponse.Rejected {
    reason: string    // 拒绝原因
}
```

## 5. 核心数据类型定义

### 5.1 金额与报价类型

#### Decimal - 自定义十进制数

T-0 Network 使用自定义 Decimal 类型避免浮点精度问题：

```
Decimal {  
    unscaled: int64    // 未缩放的整数值  
    exponent: int32    // 10 的指数（负数表示小数位）  
}  
  
// 示例：123.45 = unscaled=12345, exponent=-2  
// 计算公式：value = unscaled * 10^exponent
```

#### PaymentAmount - 支付金额

```
PaymentAmount {  
    value: Decimal    // 金额值  
}
```

#### QuoteId - 报价标识

```
QuoteId {  
    value: string      // 唯一字符串标识  
}
```

#### QuoteType - 报价类型

```
QuoteType {  
    QUOTE_TYPE_UNSPECIFIED = 0  
    QUOTE_TYPE_PAY_IN = 1      // 收款报价  
    QUOTE_TYPE_PAY_OUT = 2     // 付款报价  
}
```

## UpdateQuoteRequest - 报价推送（BAXS 作为 Provider）

```
UpdateQuoteRequest {
  quotes: repeated UpdateQuoteRequest.Quote
}

UpdateQuoteRequest.Quote {
  quote_id: QuoteId
  quote_type: QuoteType
  currency: string                // 目标货币
  bands: repeated UpdateQuoteRequest.Quote.Band // 分档报价
  expiration: Timestamp           // 过期时间
}

UpdateQuoteRequest.Quote.Band {
  max_amount: Decimal             // 该档最大金额
  settlement_amount: Decimal      // 结算金额
  payout_amount: Decimal          // 付款金额
}
```

## 5.2 区块链与稳定币枚举

表5-1 区块链枚举定义

| 枚举值                    | 编号  | 说明    | USDT 合约地址                                  |
|------------------------|-----|-------|--------------------------------------------|
| BLOCKCHAIN_UNSPECIFIED | 0   | 未指定   | -                                          |
| BLOCKCHAIN_BSC         | 10  | 币安智能链 | 0x55d398326f99059fF775485246999027B3197955 |
| BLOCKCHAIN_ETH         | 20  | 以太坊   | 0xdAC17F958D2ee523a2206206994597C13D831ec7 |
| BLOCKCHAIN_TRON        | 100 | 波场    | TR7NHqjeKQxGTCi8q8ZY4pL8otSzgJLj6t         |

表5-2 稳定币枚举定义

| 枚举值                    | 编号 | 说明  |
|------------------------|----|-----|
| STABLECOIN_UNSPECIFIED | 0  | 未指定 |

## 5.3 Travel Rule (IVMS101) 数据

T-0 Network 采用 FATF IVMS101 标准进行 Travel Rule 数据交换。以下是核心数据结构：

### Person - 人员（自然人或法人）

```
ivms101.Person {
  naturalPerson: NaturalPerson    // 自然人（二选一）
  legalPerson: LegalPerson        // 法人（二选一）
}

// 自然人
ivms101.NaturalPerson {
  name: NaturalPersonName          // 姓名
  geographicAddress: repeated Address // 地理地址
  nationalIdentification: NationalIdentification // 国民身份证
  dateAndPlaceOfBirth: DateAndPlaceOfBirth // 出生日期和地点
  countryOfResidence: string        // 居住国家
}

// 法人
ivms101.LegalPerson {
  name: LegalPersonName            // 名称
  geographicAddress: repeated Address // 地理地址
  nationalIdentification: NationalIdentification // 法人识别号
  countryOfRegistration: string     // 注册国家
}

// 地址
ivms101.Address {
  addressType: AddressTypeCode // 地址类型（GEOG=地理地址）
  streetName: string
  buildingNumber: string
  postCode: string
  townName: string
  countrySubDivision: string // 省/州
  addressLine: repeated string // 自由格式地址行
  country: string            // ISO-3166-1 alpha-2
}
```



必填字段约束

表5-3 Travel Rule 必填字段约束

| 角色       | 必填字段                                                                       |
|----------|----------------------------------------------------------------------------|
| 发起方（自然人） | name + (geographicAddress 或 dateAndPlaceOfBirth 或 nationalIdentification)  |
| 发起方（法人）  | name + (geographicAddress(GEOG) 或 nationalIdentification 或 customerNumber) |
| 受益方      | name（至少一个标识符）                                                              |

## 6. 幂等性实现规范

T-0 Network 采用 "at-least-once delivery + idempotent receivers = exactly-once processing" 的设计哲学。网络在连接断开时会使用指数退避重试，因此 BAXS 的 Provider 服务端必须正确处理重复请求。

### 三条核心规则

#### 规则 1：返回原始响应

收到已处理标识符的请求时，返回存储的原始响应，而非错误。

**错误做法：**返回 `{"failed": {"details": "Payment has already been processed"}}` — 网络将此视为真实失败，导致交易卡住。

**正确做法：**查找 `payment_id`，找到 AML 审核状态，返回 `ManualAmlCheck` 让流程继续。

#### 规则 2：等待处理完成

原始请求仍在处理中时收到重复请求，等待原请求完成，然后返回相同响应给两个请求。

**错误做法：**返回 `{"failed": {"details": "Request is already being processed"}}` — 网络视为失败并中止。

**正确做法：**等待处理完成，返回相同结果。

#### 规则 3：永不将重复视为错误

重复请求是网络重试或消息重新投递的正常事件。返回错误会破坏重试契约，导致级联故障。

### 幂等性级别

表6-1 方法幂等性分类

| 级别 | 说明 | 方法示例 |
|----|----|------|
|----|----|------|

## 6. 幂等性实现规范

|                        |                     |                                       |
|------------------------|---------------------|---------------------------------------|
| <b>IDEMPOTENT</b>      | 改变状态的请求，必须基于业务标识符去重 | CreatePayment, PayOut, FinalizePayout |
| <b>NO_SIDE_EFFECTS</b> | 只读请求，无需去重，可安全缓存     | GetQuote                              |

---

## BAXS 幂等性实现架构

```

// 幂等性存储层 - 使用 Redis + PostgreSQL
interface IdempotencyStore {
    // 检查是否已处理，返回存储的响应
    getResponse(businessId: string): Promise<StoredResponse | null>;

    // 存储响应（带 TTL）
    storeResponse(businessId: string, response: any, ttlHours: number): Promise<void>;

    // 标记处理中（防止并发处理同一请求）
    markInFlight(businessId: string, timeoutMs: number): Promise<boolean>;

    // 等待处理完成
    waitForCompletion(businessId: string, maxWaitMs: number): Promise<StoredResponse>;
}

// 实现示例
class RedisIdempotencyStore implements IdempotencyStore {
    async handleRequest<T>(<
        businessId: string,
        handler: () => Promise<T>
    >): Promise<T> {
        // 1. 检查是否已完成
        const stored = await this.getResponse(businessId);
        if (stored) return stored.response;

        // 2. 尝试标记处理中
        const acquired = await this.markInFlight(businessId, 30000);
        if (!acquired) {
            // 3. 其他实例正在处理，等待完成
            return this.waitForCompletion(businessId, 30000);
        }

        try {
            // 4. 执行业务逻辑
            const result = await handler();
            // 5. 存储结果（保留 90 天满足监管要求）
            await this.storeResponse(businessId, result, 24 * 90);
            return result;
        } catch (error) {
            // 6. 清除处理中标记，让重试可以再次执行

```

```
// 幂等性存储层 - 使用 Redis + PostgreSQL
interface IdempotencyStore {
  // 检查是否已处理, 返回存储的响应
  getResponse(businessId: string): Promise<StoredResponse | null>;

  // 存储响应 (带 TTL)
  storeResponse(businessId: string, response: any, ttlHours: number): Promise<void>;

  // 标记处理中 (防止并发处理同一请求)
  markInFlight(businessId: string, timeoutMs: number): Promise<boolean>;

  // 等待处理完成
  waitForCompletion(businessId: string, maxWaitMs: number): Promise<StoredResponse>;
}

// 实现示例
class RedisIdempotencyStore implements IdempotencyStore {
  async handleRequest<T>(<
    businessId: string,
    handler: () => Promise<T>
  ): Promise<T> {
    // 1. 检查是否已完成
    const stored = await this.getResponse(businessId);
    if (stored) return stored.response;

    // 2. 尝试标记处理中
    const acquired = await this.markInFlight(businessId, 30000);
    if (!acquired) {
      // 3. 其他实例正在处理, 等待完成
      return this.waitForCompletion(businessId, 30000);
    }

    try {
      // 4. 执行业务逻辑
      const result = await handler();
      // 5. 存储结果 (保留 90 天满足监管要求)
      await this.storeResponse(businessId, result, 24 * 90);
      return result;
    } catch (error) {
      // 6. 清除处理中标记, 让重试可以再次执行
      await this.clearInFlight(businessId);
      throw error;
    }
  }
}
```

## 7. Sandbox 测试方案

### 7.1 测试环境配置

#### 环境信息

表7-1 Sandbox 环境配置

| 配置项     | Sandbox 环境                                |
|---------|-------------------------------------------|
| API 端点  | https://api-sandbox.t-0.network           |
| 区块链网络   | Goerli (ETH) / Nile (Tron) / Chapel (BSC) |
| USDT 合约 | 测试网合约地址                                   |
| 认证方式    | 与生产环境相同的 ECDSA 签名                         |
| SDK 支持  | Go, TypeScript, Python, Java, C#          |

#### 接入前准备

##### 1. 密钥生成

- 生成 secp256k1 密钥对（专用于 Sandbox）
- 将公钥注册到 T-0 Sandbox 环境

##### 2. SDK 初始化

```
# TypeScript 示例
npx @t-0/create-provider-sdk my-adapter
cd my-adapter
# 配置 sandbox 端点和私钥
```

##### 3. 环境变量配置

```
T0_API_ENDPOINT=https://api-sandbox.t-0.network
T0_PRIVATE_KEY=0x...          # Sandbox 专用私钥
T0_PUBLIC_KEY=0x...           # 对应公钥
T0_PROVIDER_ID=...            # Sandbox 分配的 Provider ID
```

## 7.2 测试用例设计

### Phase 1: 认证与连通性测试

表7-2 认证测试用例

| 用例 ID    | 测试内容                    | 预期结果         | 优先级 |
|----------|-------------------------|--------------|-----|
| AUTH-001 | 使用正确密钥签名调用 GetQuote     | 返回 200 及正常响应 | P0  |
| AUTH-002 | 使用错误私钥签名                | 返回 401 认证失败  | P0  |
| AUTH-003 | 时间戳超出 $\pm 1$ 分钟窗口      | 返回 401 时间戳无效 | P0  |
| AUTH-004 | 未提供 X-Public-Key Header | 返回 400 缺少参数  | P1  |
| AUTH-005 | 使用未注册的公钥                | 返回 403 公钥未授权 | P1  |

### Phase 2: NetworkService API 测试（OFI 角色）

表7-3 NetworkService 测试用例

| 用例 ID   | 测试内容                                | 预期结果                               | 优先级 |
|---------|-------------------------------------|------------------------------------|-----|
| NET-001 | GetQuote - 正常询价 HKD 50000           | 返回成功报价, 含 quote_id/rate/expiration | P0  |
| NET-002 | GetQuote - 不支持的货币                   | 返回 REASON_CURRENCY_NOT_SUPPORTED   | P0  |
| NET-003 | GetQuote - 金额超出所有 Provider 限额       | 返回 REASON_LIMIT_EXCEEDED           | P0  |
| NET-004 | CreatePayment - 使用有效 quote_id       | 返回 Accepted, 含 payment_id          | P0  |
| NET-005 | CreatePayment - 不指定 quote_id (自动选择) | 返回 Accepted, 自动匹配最优报价              | P0  |
| NET-006 | CreatePayment - 使用过期的 quote_id      | 返回 REASON_QUOTE_EXPIRED            | P0  |

|         |                                      |                   |    |
|---------|--------------------------------------|-------------------|----|
| NET-007 | CreatePayment - 重复 payment_client_id | 返回原始响应（幂等性）       | P0 |
| NET-008 | FinalizePayout - 报告成功                | 返回成功确认            | P0 |
| NET-009 | FinalizePayout - 报告失败                | 返回成功确认，释放信用额度     | P0 |
| NET-010 | CompleteManualAmlCheck - 批准          | 返回含更新金额的 Approved | P1 |

### Phase 3: ProviderService 回调测试

表7-4 ProviderService 测试用例

| 用例 ID   | 测试内容                           | 预期结果                                       | 优先级 |
|---------|--------------------------------|--------------------------------------------|-----|
| PRO-001 | PayOut - 收到付款请求，正常处理           | 返回 Accepted，执行本地银行转账                       | P0  |
| PRO-002 | PayOut - 收到重复 payment_id       | 返回原始响应（不重复付款）                              | P0  |
| PRO-003 | PayOut - 收到处理中请求的重复            | 等待原请求完成，返回相同结果                             | P0  |
| PRO-004 | PayOut - 不支持的货币                | 返回 Failed<br>REASON_CURRENCY_NOT_SUPPORTED | P0  |
| PRO-005 | PayOut - 触发 AML 人工审核           | 返回 ManualAmlCheck                          | P1  |
| PRO-006 | UpdatePayment - 收到 Accepted 通知 | 正确更新本地订单状态                                 | P0  |
| PRO-007 | UpdateLimit - 收到额度更新           | 正确更新可用额度缓存                                 | P0  |
| PRO-008 | AppendLedgerEntries - 收到账本更新   | 正确入账，更新余额                                  | P1  |

### Phase 4: 端到端集成测试

表7-5 端到端测试用例



| 用例 ID   | 测试内容                     | 预期结果               | 优先级 |
|---------|--------------------------|--------------------|-----|
| E2E-001 | 完整换币流程：USDT→CAD（Interac） | 全流程 3 分钟内完成，CAD 到账 | P0  |
| E2E-002 | 完整换币流程：USDT→HKD（RTGS）    | 全流程 3 分钟内完成，HKD 到账 | P0  |
| E2E-003 | 完整换币流程：USDT→USD（Wire）    | 全流程完成，USD 到账       | P0  |
| E2E-004 | 网络中断后重试（模拟连接丢失）          | 交易最终成功，无重复付款       | P0  |
| E2E-005 | 高并发：10 笔同时交易             | 全部成功，无冲突/重复        | P1  |
| E2E-006 | 长时间运行：24 小时持续交易          | 无内存泄漏，连接稳定         | P1  |

## 7.3 端到端测试流程

端到端测试脚本 (TypeScript) :

```
import { describe, it, expect } from 'vitest';
import { T0Client } from './t0-client';

describe('E2E Exchange Flow', () => {
  const client = new T0Client({
    endpoint: 'https://api-sandbox.t-0.network',
    privateKey: process.env.T0_PRIVATE_KEY!,
  });

  it('E2E-001: Complete USDT to CAD exchange', async () => {
    // Step 1: Get Quote
    const quote = await client.getQuote({
      amount: { unscaled: 10000n, exponent: 0 },
      currency: 'CAD',
    });
    expect(quote.success).toBeDefined();
    expect(quote.success!.quoteId).toBeDefined();

    // Step 2: Create Payment
    const payment = await client.createPayment({
      paymentClientId: `test_${Date.now()}`,
      amount: { value: { unscaled: 10000n, exponent: 0 } },
      currency: 'CAD',
      paymentDetails: {
        beneficiary: {
          name: { name: 'Test Corp' },
          account: { accountNumber: '0030010123456789' },
          accountType: 'ACCOUNT_TYPE_BUSINESS',
        },
        paymentMethod: { type: 'PAYMENT_METHOD_BANK_TRANSFER' },
      },
      quoteId: quote.success!.quoteId,
      travelRuleData: generateMockTravelRule(),
    });
    expect(payment.accepted).toBeDefined();
    const paymentId = payment.accepted!.paymentId;
```

端到端测试脚本 (TypeScript) :

```
import { describe, it, expect } from 'vitest';
import { T0Client } from './t0-client';

describe('E2E Exchange Flow', () => {
  const client = new T0Client({
    endpoint: 'https://api-sandbox.t-0.network',
    privateKey: process.env.T0_PRIVATE_KEY!,
  });

  it('E2E-001: Complete USDT to CAD exchange', async () => {
    // Step 1: Get Quote
    const quote = await client.getQuote({
      amount: { unscaled: 10000n, exponent: 0 },
      currency: 'CAD',
    });
    expect(quote.success).toBeDefined();
    expect(quote.success!.quoteId).toBeDefined();

    // Step 2: Create Payment
    const payment = await client.createPayment({
      paymentClientId: `test_${Date.now()}`,
      amount: { value: { unscaled: 10000n, exponent: 0 } },
      currency: 'CAD',
      paymentDetails: {
        beneficiary: {
          name: { name: 'Test Corp' },
          account: { accountNumber: '0030010123456789' },
          accountType: 'ACCOUNT_TYPE_BUSINESS',
        },
        paymentMethod: { type: 'PAYMENT_METHOD_BANK_TRANSFER' },
      },
      quoteId: quote.success!.quoteId,
      travelRuleData: generateMockTravelRule(),
    });
    expect(payment.accepted).toBeDefined();
    const paymentId = payment.accepted!.paymentId;

    // Step 3: Monitor blockchain confirmation
    await waitForSettlement(paymentId, { timeout: 120000 });
  });
});
```

# 8. 错误处理与监控

## 错误分类与处理策略

表8-1 错误处理矩阵

| 错误类型     | 示例                        | 重试策略           | 告警级别     |
|----------|---------------------------|----------------|----------|
| 网络超时     | connect ETIMEDOUT         | 指数退避，最多 5 次    | Warning  |
| 认证失败     | 401 Unauthorized          | 不自动重试，检查密钥     | Critical |
| 时间戳无效    | timestamp too old         | 同步时钟后重试 1 次    | Warning  |
| 额度不足     | REASON_LIMIT_EXCEEDED     | 不自动重试，通知充值     | Info     |
| 无可报价     | REASON_NO_QUOTE_AVAILABLE | 30 秒后重试，最多 3 次 | Warning  |
| 服务端错误    | 500 Internal Error        | 指数退避，最多 3 次    | Error    |
| gRPC 流中断 | stream reset              | 自动重连，保持状态      | Warning  |

监控指标

核心监控指标：

```
// 业务指标
- t0_quote_latency_ms           // 报价延迟（目标 < 50ms p99）
- t0_payment_success_rate       // 支付成功率（目标 > 99.9%）
- t0_payout_execution_time_ms   // 付款执行时间
- t0_settlement_confirmation_ms // 结算确认时间

// 系统指标
- t0_active_connections         // 活跃连接数
- t0_stream_reconnect_count     // 流重连次数
- t0_signature_failures         // 签名失败次数
- t0_idempotent_dedup_count     // 幂等去重次数

// 财务指标
- t0_credit_usage_ratio         // 信用额度使用率
- t0_daily_volume_usdt         // 日交易量
- t0_failed_payout_amount       // 失败付款金额
```

告警规则

表8-2 告警规则配置

| 告警名称    | 触发条件                   | 通知方式              | 响应时间  |
|---------|------------------------|-------------------|-------|
| 认证失败激增  | 5 分钟内 > 10 次 401       | PagerDuty + Slack | 5 分钟  |
| 支付成功率下降 | 成功率 < 99% 持续 5 分钟      | PagerDuty + Slack | 10 分钟 |
| 报价延迟过高  | p99 > 200ms 持续 5 分钟    | Slack             | 30 分钟 |
| 额度即将耗尽  | 使用率 > 80%              | Slack + 邮件        | 1 小时  |
| 流连接中断   | UpdateQuote 流断开 > 30 秒 | Slack             | 15 分钟 |

## 9. 实施路线图

表9-1 集成实施路线图

| 阶段                  | 周期        | 目标                                          | 交付物                  |
|---------------------|-----------|---------------------------------------------|----------------------|
| Phase 1: 环境准备       | 第 1-2 周   | Sandbox 接入、密钥生成、SDK 初始化                     | Sandbox 连通性验证、签名测试通过 |
| Phase 2: API 实现     | 第 3-6 周   | 实现所有 NetworkService 调用 + ProviderService 回调 | 接口代码、单元测试通过          |
| Phase 3: 幂等性        | 第 7-8 周   | 幂等性存储层、重复请求处理                               | 幂等性测试通过、并发测试通过       |
| Phase 4: Sandbox 测试 | 第 9-12 周  | 全部测试用例通过、端到端流程验证                            | 测试报告、问题修复            |
| Phase 5: 安全审计       | 第 13-14 周 | 签名安全、密钥管理、传输加密审计                            | 安全审计报告               |
| Phase 6: 生产上线       | 第 15-16 周 | 生产环境部署、灰度发布、监控配置                            | 生产系统、运维手册            |

## 10. 附录

### 附录 A：术语表

| 术语            | 说明                                                  |
|---------------|-----------------------------------------------------|
| Connect RPC   | 支持 gRPC 和 REST/JSON 的 RPC 框架                        |
| ECDSA         | 椭圆曲线数字签名算法                                          |
| Keccak-256    | 以太坊使用的哈希算法（SHA-3 变体）                                |
| secp256k1     | 比特币/以太坊使用的椭圆曲线                                      |
| IVMS101       | InterVASP Messaging Standard, FATF Travel Rule 数据标准 |
| Little-endian | 低位字节存储在低地址的字节序                                      |
| Oneof         | Protobuf 中只能设置一个字段的联合类型                             |
| Unary         | 单次请求-响应的 RPC 模式                                     |
| Streaming     | 持久连接的流式 RPC 模式                                      |
| Last Look     | 执行前最后的报价确认机制                                        |

### 附录 B：参考资源

1. T-0 Network 官方文档: <https://docs.t-0.network/>
2. T-0 Provider SDK: <https://github.com/t-0-network/provider-sdk>
3. Connect RPC 框架: <https://connectrpc.com/>
4. Protobuf 语言指南: <https://protobuf.dev/programming-guides/proto3/>
5. IVMS101 规范: <https://intervasp.org/>
6. FATF Travel Rule 指南: <https://www.fatf-gafi.org/virtualassets.html>
7. Ethereum ECDSA: <https://ethereum.org/en/developers/docs/accounts/>

## 附录 C: Protobuf 文件清单

核心 Protobuf 定义文件:

- network.proto // NetworkService 接口 (BAXS 调用)
- provider.proto // ProviderService 接口 (BAXS 实现)
- payment.proto // 支付相关消息类型
- common.proto // Decimal、Blockchain、Stablecoin 等通用类型
- ivms101.proto // Travel Rule 数据结构
- ivms\_enum.proto // Travel Rule 枚举类型

### 文档控制信息

文档版本: v1.0 | 最后更新: 2026年7月1日 | 下次审查: 2026年10月1日

文档所有者: BAXS 技术团队 | 审批状态: 待审批

本文件包含内部技术信息, 未经授权不得分发。